

Health Management & Resource Governance

Liveness / readiness / startup probes, requests & limits, ResourceQuota, LimitRange, and QoS classes - verified with real kernel-level evidence.

LIVENESS

READINESS

STARTUP

LIMITS

QOS CLASS



Where this sits in the course

Day 01-02

Workloads, config,
Secrets, RBAC.

Day 03-04

Services, DNS,
NetworkPolicy,
and what's
underneath.

Day 05

Ingress, TLS,
load balancing,
egress.

Day 06

PersistentVolumes,
real NFS,
a real CSI driver.

Today

Whether a workload is healthy, and how much of the machine it's allowed to use.

```
kubectl describe node | grep -A6 Allocatable  
kubectl top node
```

What this day is really about

Health checks (probes)

Is this container doing its job right now?

liveness / readiness / startup

Resource governance

How much CPU/memory is it allowed to use, and what happens when it crosses that?

requests / limits / quota / QoS

Fully real on one node

Probes, cgroup limits, ResourceQuota, and QoS classes are per-Pod/per-namespace mechanics - none of them need a second node. The one thing this module deliberately does NOT do: starve the node itself to trigger a real node-pressure eviction - that would affect every other lab and service sharing this VM. The mechanism behind eviction order (`oom_score_adj`) is still shown for real, per Pod.

Probes: three types, one job each

Probe	Question it answers	Failure consequence
Liveness	Is this container still working?	kubelet KILLS and RESTARTS the container
Readiness	Can it handle traffic right now?	Pod pulled from Service ENDPOINTS - keeps running
Startup	Has this slow container finished starting?	liveness/readiness NOT evaluated while running

All three use the same check types (httpGet, exec, tcpSocket, grpc) and the same tuning knobs (initialDelaySeconds, periodSeconds, failureThreshold). Only the consequence differs.

Liveness probe: kills and restarts

```
livenessProbe:
  exec:
    command: ["cat", "/tmp/healthy"]
  periodSeconds: 5
  failureThreshold: 1
```

Genuinely healthy for 30s, then silently broken forever. Only the probe notices.

```
$ kubectl get pod liveness-demo
NAME          READY   RESTARTS   AGE
liveness-demo 1/1      1 (14s ago) 80s

$ kubectl describe pod liveness-demo
Warning Unhealthy Liveness probe failed: cat: can't open '/tmp/healthy'...
Normal Killing Container app failed liveness probe, will be restarted
```

RESTARTS went 0 -> 1 - not a crash, not an OOM: a liveness failure kubelet chose to act on.

Readiness probe: reroutes, doesn't restart

```
$ kubectl exec web-...-7nrdp -- rm /tmp/ready
$ kubectl get pods -l app=web
NAME           READY   RESTARTS   AGE      # NOT restarted
web-...-7nrdp   0/1     0          19s
web-...-sg52l   1/1     0          20s

$ kubectl get endpointslice ... -o jsonpath='{...}'
{"addresses":["10.42.0.170"],"conditions":{"ready":false}}
{"addresses":["10.42.0.169"],"conditions":{"ready":true}}
```

5 requests through the Service after the flip: all 5 succeeded, none routed to the unready Pod - the same KUBE-SEP-* iptables mechanism from Day 04, just missing an entry for it.

Startup probe: protects a slow starter

Two Pods, IDENTICAL 20s-to-heal container, IDENTICAL liveness check. Only one has a startupProbe.

Pod	RESTARTS after 105s	Events
slow-starter-bad	2 (climbing)	Liveness probe failed -> Killing, every time
slow-starter-good	0	Startup probe failed x4, then succeeds - no Killing, ever

bad's liveness fires at t~5s, before the file exists at t=20s - and every restart resets its own 20s clock, so it can never finish. good's startupProbe (30s budget) absorbs those same early failures without ever restarting the container.

Requests vs. Limits: the mental model

requests -> what the SCHEDULER reserves (decides WHICH node it can run on)

limits -> what the KERNEL enforces at runtime (decides what happens using MORE)

No requests/limits at all -> can burst to use anything free, but first in line to be killed under pressure.

requests too high -> the Pod can't be scheduled anywhere - a capacity problem, not an app bug.

limits too low -> the app runs but gets throttled (CPU) or killed (memory) - very different failure modes.

Requests drive scheduling

```
$ kubectl apply -f ex04-requests-limits/unschedulable.yaml # requests cpu: "10"
$ kubectl get pod too-big-to-schedule
NAME                READY   STATUS
too-big-to-schedule  0/1     Pending

$ kubectl describe pod too-big-to-schedule
Warning  FailedScheduling  0/1 nodes are available: 1 Insufficient cpu.
```

This node has 4 allocatable CPUs. Asking for 10 isn't "slow" or "stuck" - Insufficient cpu is the entire, permanent story. No amount of waiting fixes it; only lowering the request (or adding capacity) does.

CPU limits: throttled, not killed

```
$ kubectl top pod cpu-throttle-demo
NAME                CPU(cores)
cpu-throttle-demo    200m          # pinned exactly at the limit
```

```
$ kubectl exec cpu-throttle-demo -- cat /sys/fs/cgroup/cpu.max
20000 100000      # 20ms quota / 100ms period = 0.2 CPU = 200m

$ kubectl exec cpu-throttle-demo -- cat /sys/fs/cgroup/cpu.stat
nr_periods 170
nr_throttled 167    # 167 of 170 windows throttled
                    throttled_usec 3314146
```

limits.cpu becomes a literal cpu.max quota the kernel's CFS bandwidth controller enforces. Nothing crashes; it's just paused.

Memory limits: killed, not throttled

```
$ kubectl get pod oom-kill-demo
NAME          READY   STATUS
oom-kill-demo 0/1     OOMKilled

$ kubectl get pod oom-kill-demo -o jsonpath='{...state.terminated}'
{"exitCode": 137, "reason": "OOMKilled", "startedAt": "...54Z", "finishedAt": "...58Z"}
```

exitCode 137 = 128 + 9 (SIGKILL) from the kernel OOM-killer, the instant the cgroup crossed its 20Mi limit writing to /dev/shm (tmpfs - real, UNSWAPPABLE memory; swap is off on this node). There is no "slow down" for memory the way there is for CPU. Writing to a regular file first was tried and did NOT OOM - disk-backed page cache is reclaimable, so the kernel wrote it out instead of killing anything.

ResourceQuota: a hard ceiling on the namespace

```
apiVersion: v1
kind: ResourceQuota
spec:
  hard:
    pods: "3"
    requests.cpu: "500m"
    requests.memory: 256Mi
    limits.cpu: "1"
    limits.memory: 512Mi
```

```
$ kubectl describe resourcequota team-quota
Resource      Used      Hard
pods          3         3

$ kubectl run bare-pod-4 \
  --image=busybox:1.36 -- sleep 3600
Error: pods "bare-pod-4" is forbidden:
exceeded quota: team-quota,
requested: pods=1, used: pods=3,
limited: pods=3
```

Enforced by the API SERVER at admission time - the 4th Pod is rejected before it's ever created, not created-then-stuck.

LimitRange: per-container defaults and bounds

```
apiVersion: v1
kind: LimitRange
spec:
  limits:
  - type: Container
    default:
      cpu: 100m, memory: 64Mi
    defaultRequest:
      cpu: 50m, memory: 32Mi
    min: { cpu: 10m, memory: 8Mi }
    max: { cpu: 500m, memory: 256Mi }
```

```
$ kubectl run bare-pod ... -- sleep 3600
# NO resources: block at all
$ kubectl get pod bare-pod -o jsonpath=\
  '{...resources}'
limits: {cpu: 100m, memory: 64Mi}
requests: {cpu: 50m, memory: 32Mi}

$ kubectl run too-big ... (limits.memory: 1Gi)
Error: max memory per Container is 256Mi,
      but limit is 1Gi
```

ResourceQuota and LimitRange compose: the quota (Slide 12) counted whatever the LimitRange already filled in.

QoS classes: derived, not declared

Class	How you get it	This module's Pod
Guaranteed	requests == limits, every resource, every container	qos-guaranteed
Burstable	has requests/limits, but they don't match	qos-burstable
BestEffort	no requests/limits at all	qos-besteffort

```
$ kubectl get pods -o custom-columns=NAME:.metadata.name,QOS:.status.qosClass
qos-guaranteed    Guaranteed
qos-burstable     Burstable
qos-besteffort    BestEffort
```

The real mechanism behind "BestEffort dies first"

```
$ for p in qos-guaranteed qos-burstable qos-besteffort; do
  echo "$p: $(kubectl exec $p -- cat /proc/1/oom_score_adj)"
done
qos-guaranteed: oom_score_adj=-997
qos-burstable: oom_score_adj=998
qos-besteffort: oom_score_adj=1000
```

oom_score_adj is a real Linux kernel value the kubelet sets on each container's main process, based purely on QoS class. Lower = the OOM-killer leaves it alone longer; higher = first candidate killed when the NODE runs low on memory. "BestEffort dies first" isn't a policy applied after the fact - it's a number in /proc, set at container start.

Best-practice checklist

Give every container a liveness probe checking ITS OWN health, not a dependency's - use readiness for that.

Add a startupProbe for unpredictable startup time instead of loosening the liveness timing for everyone.

Always set requests - Pods with none get BestEffort (first to die under pressure).

Set memory limits deliberately - an OOMKilled container is instant and total, no graceful degradation.

Use LimitRange so "forgot to set resources" fails safe with sane defaults, not BestEffort scheduling.

Use ResourceQuota per namespace/team so one workload can't starve every other tenant.

Troubleshooting sequence

```
kubectl describe pod <name> -n <ns>           # events + Last State - the real reason is here
kubectl get events -n <ns> --sort-by=.lastTimestamp | tail -20
kubectl top pod -n <ns>                        # needs metrics-server
          kubectl exec <pod> -n <ns> -- cat /sys/fs/cgroup/cpu.max /sys/fs/cgroup/cpu.stat
```

Symptom	Likely cause
RESTARTS climbing	liveness probe failing, or the app crashing - check Last State
Running but 0/1 Ready	readiness probe failing - check its own logic
Pending, FailedScheduling	requests exceed any node's Allocatable
OOMKilled, exitCode 137	memory limit too low, or a real leak
CPU-bound but never crashes	check cpu.stat nr_throttled/throttled_usec
Pod creation Forbidden	ResourceQuota exhausted or LimitRange violated

Hands-on labs — built and verified

Lab	Proves
ex01 liveness probe	failed check -> container killed and restarted
ex02 readiness probe	failed check -> removed from Service endpoints, no restart
ex03 startup probe	same slow starter: crash-loops without it, healthy with it
ex04 requests & limits	Insufficient cpu; cgroup throttling; OOMKilled exit 137
ex05 ResourceQuota + LimitRange	auto-filled defaults, min/max rejection, namespace pod cap
ex06 QoS classes	Guaranteed/Burstable/BestEffort + real oom_score_adj per Pod

Same pattern as Days 01-06: manifests + LAB-MANUAL.md (steps + troubleshooting) + OUTPUT.md (real captured output).

RECAP

Healthy is a probe result. Allowed is a kernel number.

Liveness restarts; readiness reroutes; startup buys a slow container time before either gets a vote.

requests are a scheduling input; limits are a runtime enforcement - CPU throttles, memory kills.

ResourceQuota caps a namespace; LimitRange defaults and bounds a container; they compose.

QoS class is derived from requests vs limits and sets a real kernel oom_score_adj.

"BestEffort dies first" is literally true in /proc, not just in the docs.